MULTI-AGENT AI SUPPORT SYSTEM

Sribhava

Resolves routine tickets on its own and sends risky ones to a human.

Sindhu Vadde



THE PROBLEM

Most support tickets are routine, but a wrong answer is expensive

Slow

Customers wait in a queue for answers that already exist in the policy.

Costly

People spend their day on repeat questions instead of the hard cases.

Risky to automate

A bot that confidently refunds the wrong amount does more harm than a slow reply.

HOW IT WORKS

Meet the four agents



Sorter

The Classifier. Reads the message and labels its category.



Scholar

The Policy agent. Looks up the rules that apply.



Doer

The Action agent. Resolves it, or escalates it.



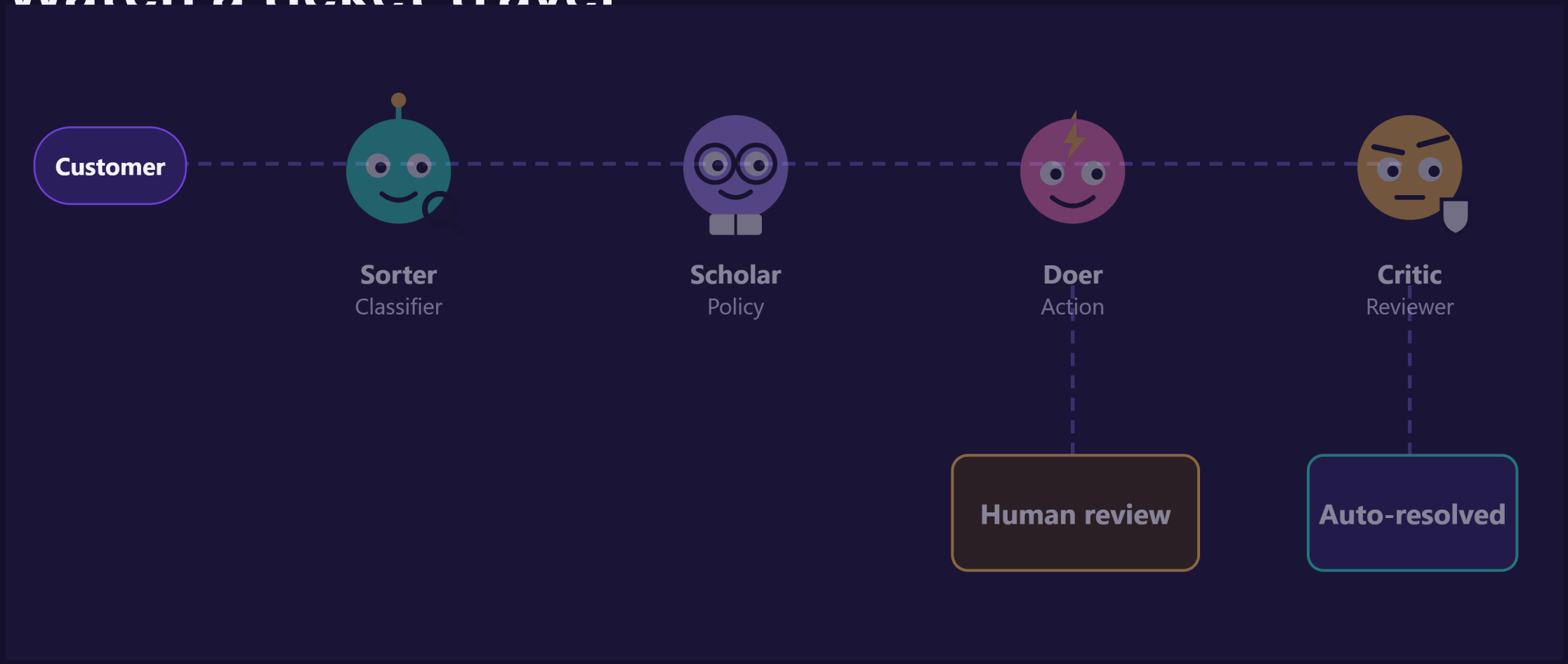
Critic

Double-checks the plan and can veto it.

Anything risky goes to a human, who approves or rejects.

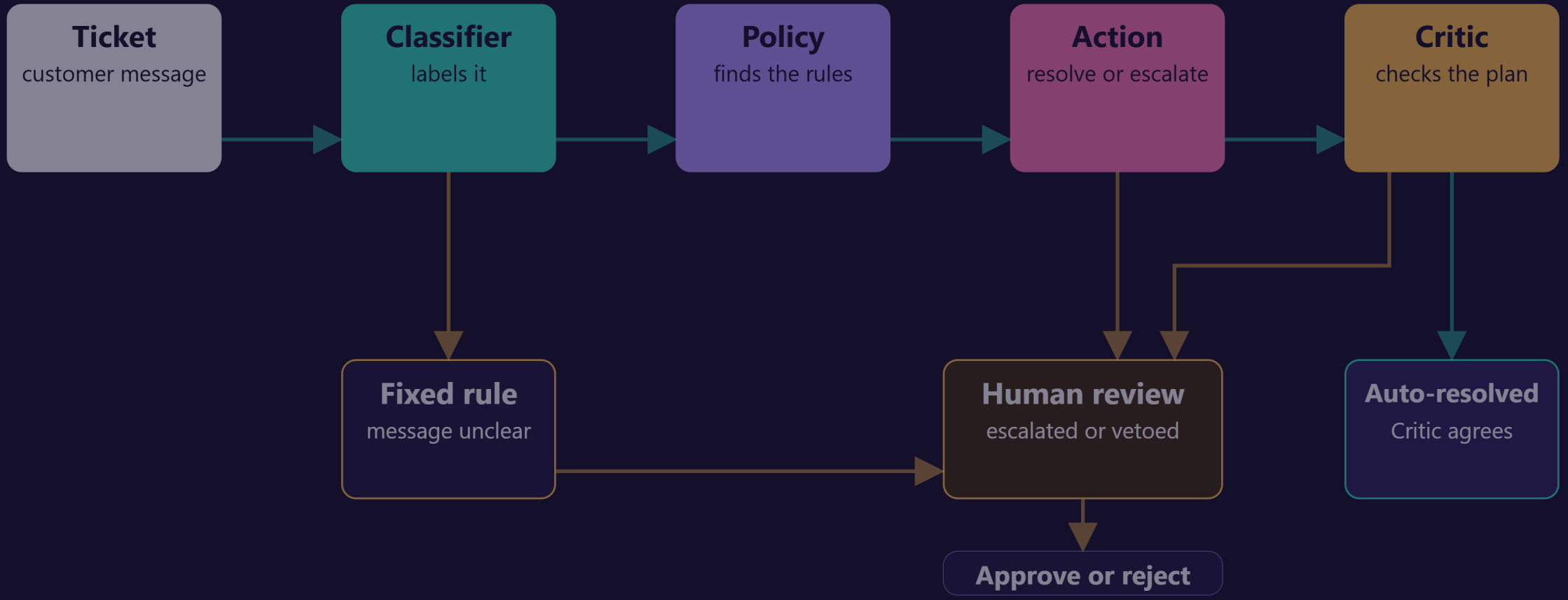
LIVE ANIMATION

Watch a ticket travel



FLOWCHART

How one ticket travels



Follow one ticket

Ticket: refund for a damaged item



Sorter



Scholar



Doer

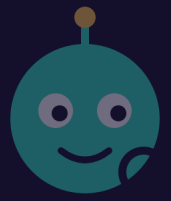


Critic

Step 1. The Sorter reads the message and labels it: category refund, intent damaged item.

Follow one ticket

Ticket: refund for a damaged item



Sorter



Scholar



Doer

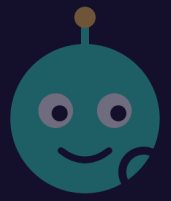


Critic

Step 2. The Scholar searches the company policy and writes down which rules apply.

Follow one ticket

Ticket: refund for a damaged item



Sorter



Scholar



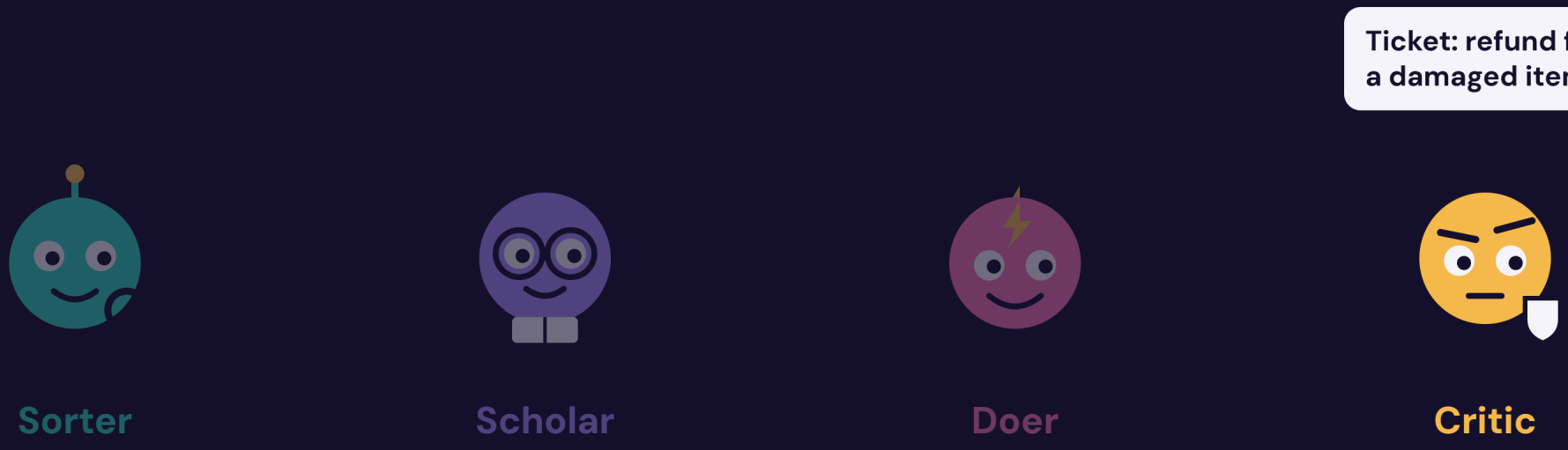
Doer



Critic

Step 3. The Doer decides: resolve it automatically, or escalate to a person.

Follow one ticket



Step 4. The Critic double-checks the plan. If an automatic answer looks unsafe, the Critic vetoes it.

Follow one ticket

Ticket: refund for a damaged item



Safe plan: auto-resolved

Risky or unclear: a human approves or rejects

Nothing leaves the system as an automatic answer unless every layer agrees.

SAFETY

Three layers stop a bad answer

1

Fixed rule

If the Classifier cannot understand the message, it always goes to a human.

2

Critic veto

A second agent reviews the plan and can block an automatic answer.

3

Human approval

Every escalated ticket waits for a person to approve or reject it.

A real bug, found and fixed: a nonsense message was once auto-resolved (ticket 94). Layer 1 is the fix.

SPEED AND COST

Measured on real runs

5.5 s

Median time per ticket

Half of all tickets finish faster than this.

7.8 s

p95 time per ticket

19 of every 20 tickets finish within this.

1,287

Tokens in per ticket

About 269 tokens come back out.

~$0.001

Estimated cost per ticket

Range $0.0005 to $0.0011. Free tier: $0.

Timing from tickets 18 onward. Cost is an estimate from token counts and published prices.

Overview: who handled each ticket

Live dashboard



Performance: where time and tokens go

Live dashboard

Deploy

Sribhava Retail

Multi-agent customer support desk · live operations view

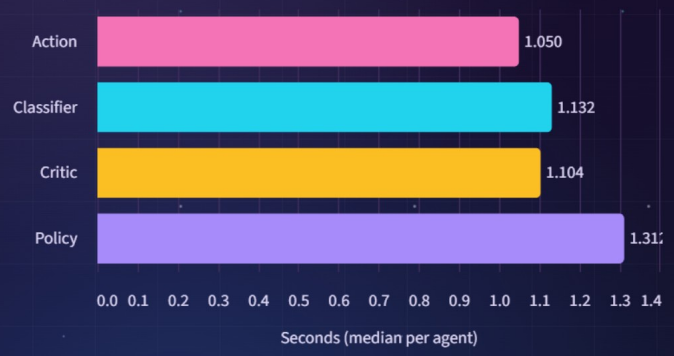
Live · refreshes every 5 seconds

Dashboard | Review queue

Overview | Performance | Safety | Trace explorer

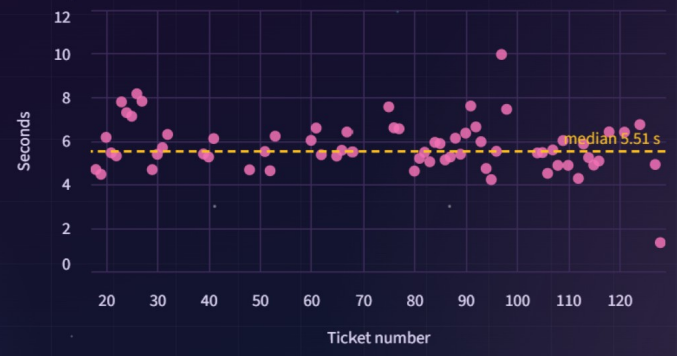
Only tickets from #18 onward are shown: earlier tickets were run with an older version that counted server start-up inside every ticket.

Where the time goes



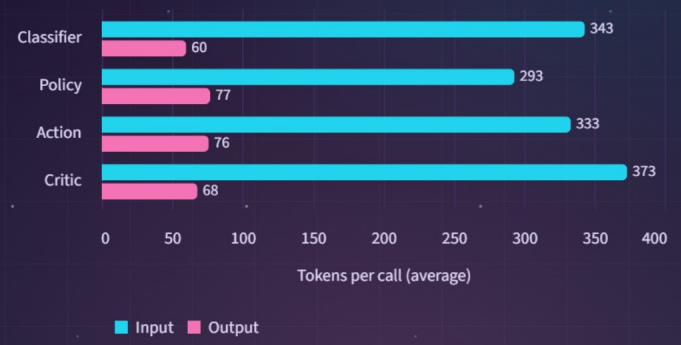
The four agents add up to about 4.60 s of the 5.51 s typical ticket; the rest is time outside the agents' own model calls (for example database and tool calls).

Time per ticket



Off the top of the chart: ticket #59 took 13 s.

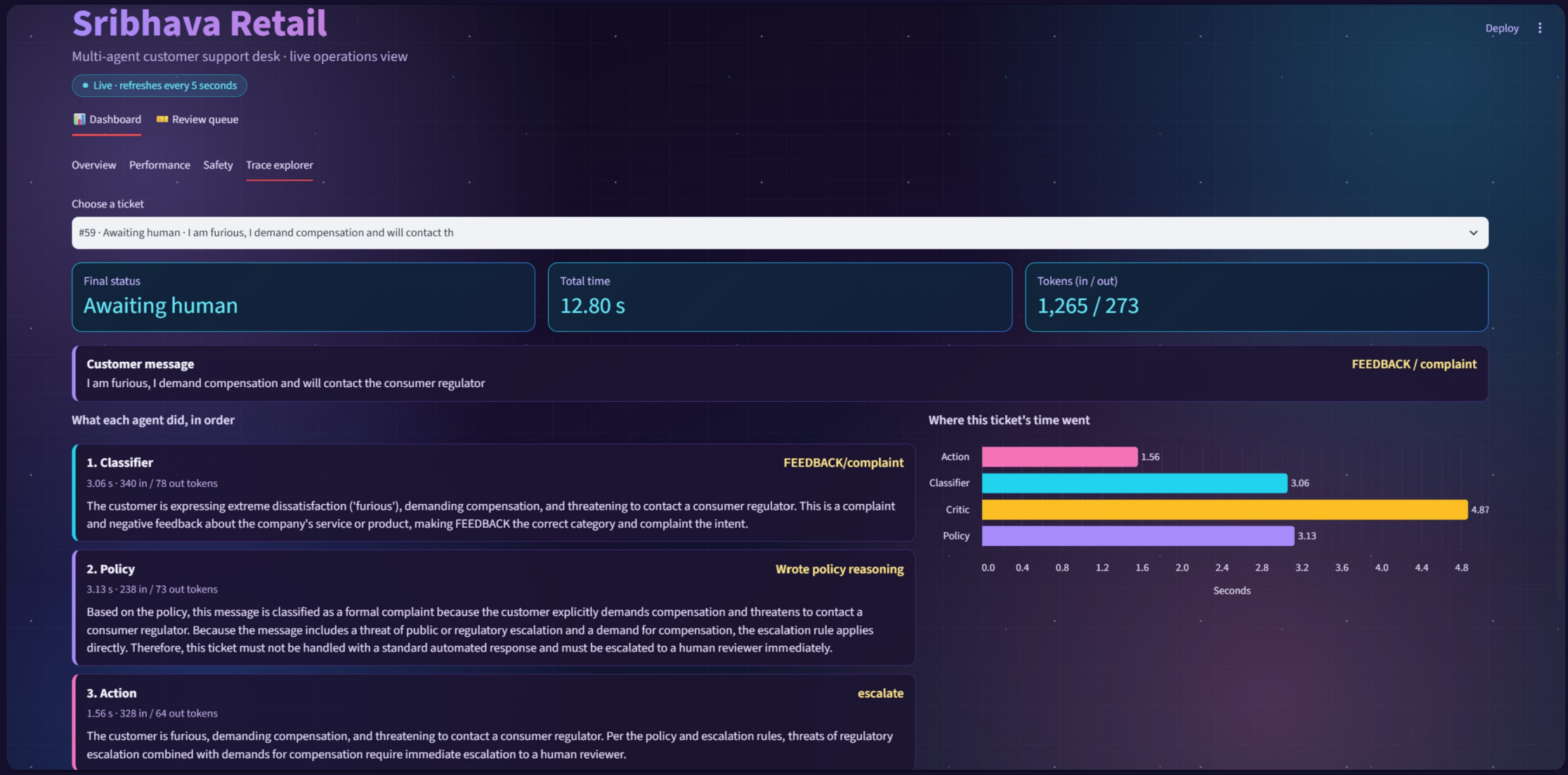
Where the tokens go



Input is what the agent reads; output is what it writes. Output costs more per token.

Trace explorer: why a ticket was escalated

Ticket 59



Safety: what the layers stopped

Live dashboard

Sribhava Retail

Deploy

Multi-agent customer support desk · live operations view

● Live · refreshes every 5 seconds

Dashboard | Review queue

Overview | Performance | Safety | Trace explorer

Three safety layers sit between a customer message and an automatic answer: (1) the Classifier can flag a message as unclassifiable, and a fixed rule then sends it to a human; (2) the Critic agent can veto an auto-resolve; (3) every escalation waits for a human to approve or reject it.

Critic overruled	Of AI auto-resolve proposals	Escalated by the fixed rule	Errors
1	2%	1	9

How each decision was reached



This counts every ticket that reached the Action step or the fixed rule (92 in total), including older, errored and unfinished ones, so it can be larger than 'Finished by the AI' on the Overview tab. A Critic disagreement on a ticket the Action agent already escalated changes nothing, so only vetoes of auto-resolves are counted as 'overruled'.

Latest tickets the safety layers stopped

#	Stopped by	Message	Reason
128	Fixed rule	asdf qwerty zxcv	Escalated by rule: the Classifier could not find a real support request in this message.
4	Critic	I want to talk to a customer service person	The customer explicitly requested to speak to a customer service person ('I want to talk to a customer service person'). While phone support might be restricted, the customer is asking for human assistance/escalation to an agent, and 'auto_resolving' a request from a customer asking for human help without actually routing them or having an agent handle it violates customer service safety and routing standards. This ticket requires human agent intervention or proper escalation, not auto-resolution.

ENGINEERING

Built and checked like a real service

What it is built with

- Gemini with function calling
- LangGraph to route between agents
- ChromaDB for policy search
- A custom MCP server for tools
- FastAPI, SQLite and Streamlit

How it is checked

- Automated tests with pytest
- CI on every push: tests and Docker build
- Retries with backoff when the model API is busy
- Read-only dashboard access to the database
- Customer text escaped before it is shown

WHAT THIS PROJECT SHOWS

Skills behind the build

Agent design

Splitting a job into narrow agents with a checker on top.

Evaluation

Measuring speed, cost and decisions, and saying what is not measured.

Safety thinking

Finding an unsafe case and fixing it with a rule that does not rely on the model.

Data and analytics

Cleaning outliers, choosing medians and p95, and building a live dashboard.

Engineering habits

Tests, CI, Docker, retries and read-only data access.

Cost awareness

Tracking tokens per ticket and estimating cost before scaling up.

HONEST LIMITS

What the numbers do and do not show

Measured

- 49 of 50 designed test tickets matched the expected policy outcome
- Time and token counts from real runs
- One unsafe case found and fixed

Not measured yet

- Real-world escalation and autonomous rates. The 50-ticket set was designed, so its split is not a real rate
- Classifier accuracy on random tickets

WHAT IS NEXT

From project to production

- Deploy to a cloud free tier, with a budget alert
- Timeouts, and detection of stuck tickets
- Health check and automatic restart of the tool server
- Postgres, login, and rate limits
- A run on random tickets for real rates

Sindhu Vadde · [GitHub link] · [Demo video link]